Cairo University
Faculty of Engineering
Electronics & Communications Dept.

NN ASSIGNMENT 2

Artificial Neural Networks

Technical Report

Prepared by:

Abdallah Essam Mohamed Mahmoud	ID: ...
Zeyad Antar Othman Abu-Zaid	ID: 9220331
Abdelrahman Hatem Nasr Youssef	ID: 9220461

Submitted to:

Prof. Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

April 2026

Contents

1 Problem 1: Training Feed-Forward Neural Networks (MLP)

1.1 Introduction

This part explores the classification of the **ReducedMNIST** handwritten digit dataset using **Multilayer Perceptrons (MLPs)**. The objective is to evaluate how *network depth* influences performance when combined with different input feature representations.

A Multilayer Perceptron is a fully connected feed-forward neural network. Every neuron in one layer connects to every neuron in the next layer. It is the simplest form of "deep" learning:

- **Input layer:** receives the feature vector (e.g. 225 DCT coefficients).
- **Hidden layers:** apply a learned linear transformation followed by a non-linear activation function (ReLU in our case). More hidden layers = deeper network = ability to learn more complex patterns.
- **Output layer:** 10 neurons (one per digit 0–9), producing class probabilities via the Softmax function.

We trained MLPs with **1, 3, and 4 hidden layers** using features derived from:

1. **Discrete Cosine Transform (DCT)** — 225 features (top-left 15×15 block).
2. **Principal Component Analysis (PCA)** — 262 components retaining 95% of variance.
3. **Autoencoder (AE)** — a neural network trained to compress images into 64 features.

The dataset is a reduced version of the standard MNIST with 10,000 training images (1,000 per digit) and 2,000 test images (200 per digit), each 28×28 pixels normalised to $[0, 1]$.

1.2 Results

System: Intel Core i7-9850H @ 2.60 GHz, 32 GB RAM, Windows 11, Python 3.12, PyTorch 2.11 (CPU only).

Hyper-parameters: Epochs = 10, Batch size = 64, Learning rate = 10^{-3}, Optimizer = Adam, Activation = ReLU, Loss = CrossEntropyLoss. Autoencoder pre-training: 20 epochs, bottleneck = 64, MSE loss.

Hidden Layers	DCT		PCA		AutoEncoder	
	Acc (%)	Time (ms)	Acc (%)	Time (ms)	Acc (%)	Time (ms)
1 layer [128]	95.8	3758.3	93.5	4470.1	96.4	3944.1
3 layers [256-128-64]	96.4	6096.5	93.8	5808.2	**97.2**	6135.4
4 layers [256-128-64-32]	95.5	7154.5	93.3	7322.9	96.7	6417.3

Table 1. Test accuracy and processing time for each feature–depth combination.

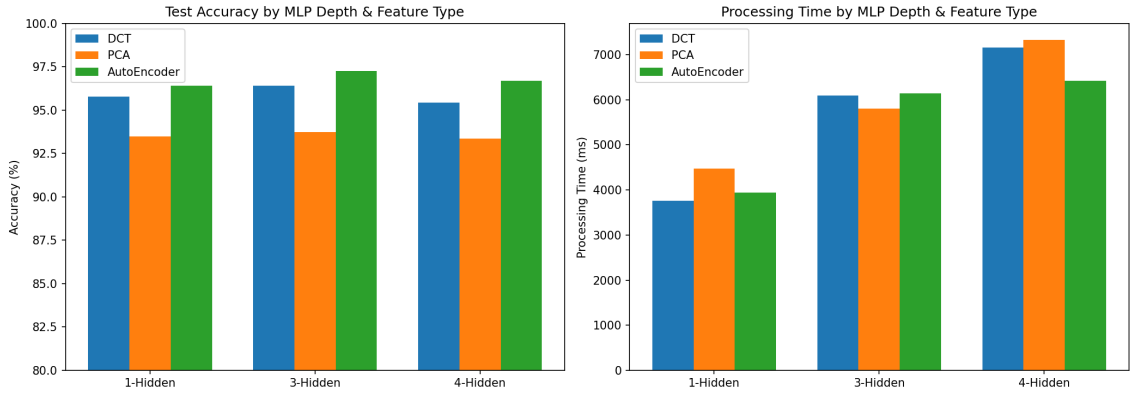

Figure 1. Comparison of test accuracy (left) and processing time (right) across all feature types and MLP depths.

1.3 Analysis and Discussion

1.3.1 Effect of Network Depth and Overfitting

Increasing depth from 1 to 3 hidden layers gave a modest accuracy boost for DCT (95.8% → 96.4%) and AutoEncoder (96.4% → 97.2%), but going to 4 hidden layers *decreased* accuracy in every feature type — DCT dropped to 95.5%, PCA to 93.3%, and AutoEncoder to 96.7%.

This accuracy drop at 4 layers is a clear sign of **overfitting**:

- **Too many parameters for the data:** the 4-layer MLP (256–128–64–32) has significantly more trainable weights than the 1-layer variant. With only 1,000 training samples per class, the extra parameters fit noise in the training set rather than learning generalisable patterns, leading to lower test accuracy.
- **Vanishing gradient effect:** deeper networks propagate smaller gradients to early layers, making it harder for those layers to learn useful features in only 10 epochs.
- **No regularisation:** the models use no dropout, weight decay, or early stopping, so deeper architectures are especially prone to memorising the training data.

- **Sweet spot at 3 layers:** the 3-layer MLP strikes the best balance — enough capacity to capture non-linear patterns without overfitting the limited dataset.

1.3.2 Effect of Feature Type

The **Autoencoder features achieved the highest accuracy** (97.2%) despite being the smallest representation (only 64 dimensions).

Ranking: AutoEncoder (97.2%) > DCT (96.4%) > PCA (93.8%)

Why AutoEncoder outperforms:

- The autoencoder learns a **non-linear** mapping, allowing it to capture complex relationships between pixels that linear PCA cannot.
- Its 64-D features are extremely compact yet highly discriminative — the bottleneck forces it to retain only the most class-relevant information.
- The autoencoder is trained *on the same data*, so its features are tailored to this specific dataset.

Why DCT outperforms PCA:

- DCT captures frequency information which is well-suited for digit recognition (global shape patterns).
- PCA with 262 components retains some noisy dimensions, which can slightly confuse the classifier.

1.3.3 Processing Time

Processing time scales with both **network depth** and **input dimensionality**:

- PCA (262-D input) consistently took the longest due to the larger first weight matrix.
- AutoEncoder (64-D input) was the fastest despite achieving the best accuracy — demonstrating that good feature engineering reduces both compute cost and error rate.
- Going from 1 to 4 hidden layers roughly doubles the processing time (e.g. DCT: 3758 → 7155 ms), while accuracy does not improve proportionally.